

# Chapter 40 - Interrupts, Raster Timing, and Polling

Intuition Engine has real interrupt lines, but it is not a machine where every event must be handled by an interrupt routine. Some chips raise CPU-visible events. Some expose a latch or a status bit. Some are best driven by a simple wait or poll from BASIC.

The practical rule is:

1. Use a device interrupt when the CPU must keep doing other work.
2. Use a status bit when the CPU is already waiting for that event.
3. Use WAIT\_VBLANK, VSYNC, or a chip raster register when the event is part of display timing.

This chapter is a timing cookbook. Chapter 31 gives the general trap and exception map. The per-chip chapters give the full register maps.

## 40.1 Interrupt Sources

The common interrupt masks on the shared bus are:

| Source  | Mask | Main use                                          |
|---------|------|---------------------------------------------------|
| VBI     | 1    | Vertical blank or frame boundary work             |
| DLI     | 2    | ANTIC display-list interrupt or raster split work |
| Blitter | 4    | VideoChip blitter completion                      |

The CPU does not see those names directly. Each CPU receives the event through its own interrupt shape:

| CPU  | Delivery shape                                                                                         |
|------|--------------------------------------------------------------------------------------------------------|
| IE64 | External interrupt delivery when interrupts are enabled, with the cause mask visible to the trap path. |
| IE32 | Single interrupt vector at memory \$0004.                                                              |
| 6502 | IRQ and NMI, depending on the source and adapter.                                                      |
| Z80  | INT or NMI, following the selected interrupt mode.                                                     |
| M68K | Auto-vectored levels. Video events use level 5; audio events use level 4.                              |
| x86  | INTR or NMI through the interrupt vector table.                                                        |

An interrupt pulse that arrives while a CPU has interrupts disabled can be dropped. A level source that remains asserted can be seen later when the CPU unmask it. This is why polling remains important: a status bit can tell you what the device is doing even when an interrupt was not taken.

## 40.2 Polling Paths

These are the common wait and poll registers:

| Register            | Address | What to test                                                |
|---------------------|---------|-------------------------------------------------------------|
| WAIT_VBLANK         | \$F2580 | Write any value to wait for the next VideoChip VBlank edge. |
| VIDEO_STATUS        | \$F0008 | Bit 0 content, bit 1 VBlank, bit 2 framebuffer error.       |
| BLT_STATUS          | \$F0044 | Bit 1 done, bit 2 error.                                    |
| ULA_STATUS          | \$F2008 | ULA status, including VBlank state in the ULA chapter.      |
| TED_V_RASTER_STATUS | \$F0F5C | TED raster compare status.                                  |
| ANTIC_NMIST         | \$F2134 | ANTIC DLI and VBI pending latches.                          |
| ANTIC_STATUS        | \$F213C | ANTIC VBlank active bit.                                    |

WAIT addr,mask[,xor] in BASIC is a polling instruction. It does not mean "sleep for this many ticks". It reads addr repeatedly until ((PEEK(addr) EOR xor) AND mask) is non-zero.

## 40.3 First Polling Example

This short listing waits for VBlank using the CPU Wait block, then changes the VideoChip fill colour and starts a small blit.

```

10 REM VBLANK THEN BLITTER
20 POKE32 &H000F0004,4
30 POKE32 &H000F0080,0
40 POKE32 &H000F0084,&H00100000
50 POKE32 &H000F0000,1
60 POKE32 &H000F2580,1
70 POKE32 &H000F0028,&H00100000
80 POKE32 &H000F002C,80
90 POKE32 &H000F0030,40
100 POKE32 &H000F003C,&H00004080
110 POKE32 &H000F0020,1
120 POKE32 &H000F001C,1
130 WAIT &H000F0044,2
140 PRINT PEEK32(&H000F0044)

```

Expected result: the programme waits for a frame boundary, fills a small VideoChip rectangle, waits until BLT\_STATUS has the done bit, and prints a value with bit 1 set.

Lines 20 to 50 select and enable a visible VideoChip mode. Line 60 waits for the next VBlank. Lines 70 to 120 set up a fill blit. Line 130 polls the blitter done bit. The programme uses polling because it has nothing useful to do while the blitter is running.

## 40.4 ANTIC DLI Showpiece

ANTIC has a display-list interrupt bit in each display-list instruction. In Intuition Engine the DLI bit sets ANTIC\_NMIST bit 7 when ANTIC\_NMIEN bit 7 is enabled, and pulses the shared DLI interrupt source.

This BASIC example uses the latch path first. It builds a tiny display list with a DLI-marked mode line, enables ANTIC, and polls NMIST.

```

10 REM ANTIC DLI LATCH
20 DL=&H00002000:SCR=&H00002100
30 POKE8 DL+0,&H70
40 POKE8 DL+1,&HC2
50 POKE8 DL+2,SCR AND 255
60 POKE8 DL+3,INT(SCR/256) AND 255
70 POKE8 DL+4,&H41
80 POKE8 DL+5,DL AND 255
90 POKE8 DL+6,INT(DL/256) AND 255
100 FOR I=0 TO 39:POKE8 SCR+I,65:NEXT
110 POKE32 &H000F2140,&H28
120 POKE32 &H000F2144,&H58
130 POKE32 &H000F2108,DL AND 255
140 POKE32 &H000F210C,INT(DL/256) AND 255
150 POKE32 &H000F2130,&H80
160 POKE32 &H000F2100,&H22
170 POKE32 &H000F2138,1
180 WAIT &H000F2134,&H80
190 PRINT "DLI ";PEEK32(&H000F2134) AND &H80
200 POKE32 &H000F2134,0

```

Expected result: ANTIC displays a text row and the programme prints DLI 128. Line 200 acknowledges the latch by writing to ANTIC\_NMIST. The value is not a selective clear mask: any write to that register clears the pending ANTIC NMI latches.

The display-list byte at line 40 is C2: display-list mode 2, with LMS and DLI bits set. The interrupt is useful when a machine-code programme wants to change colours or scroll values at that exact line. The latch is useful when BASIC only needs to prove that the line was reached.

## 40.5 M68K Handler Setup

The M68K is the most natural CPU for classic interrupt handlers in Intuition Engine. Video events use auto-vector level 5, whose vector entry is at offset \$74. Audio events use level 4, whose vector entry is at offset \$70.

To install a handler:

1. Place the handler code in RAM.
2. Store the handler address as a big-endian longword at the vector offset.
3. Set the interrupt mask in SR low enough to admit the level.
4. In the handler, acknowledge the device latch that caused the interrupt.
5. Finish with RTE.

For video, the common acknowledgement targets are ANTIC\_NMIST, BLT\_STATUS, a chip-specific raster status register, or whatever status register the owning chapter documents. Do not clear a latch you have not tested. Several devices can share the same CPU level.

## 40.6 Which Timing Method To Use

| Job                                         | Best first choice              |
|---------------------------------------------|--------------------------------|
| Start drawing after the next frame boundary | VSYNC or WAIT_VBLANK.          |
| Wait for one VideoChip blit to finish       | Poll BLT_STATUS done or error. |

| Job                                           | Best first choice                                        |
|-----------------------------------------------|----------------------------------------------------------|
| Change colours at an ANTIC display-list line  | DLI in machine code, ANTIC_NMIST polling in BASIC.       |
| Wait for TED raster compare in BASIC          | TED_V_RASTER_STATUS.                                     |
| Cycle a ULA border once per frame             | ULA VBlank status or Chapter 8's VBlank polling pattern. |
| Keep a CPU busy with game logic while waiting | Interrupt handler or split the work over frames.         |
| Debug a stuck M68K interrupt                  | IRQ diagnostics at \$F23C0 to \$F23DF.                   |

## 40.7 Side Effects and Limits

- A dropped interrupt pulse is not an error. It means the CPU was not in a state to accept that edge.
- A latched status bit remains visible until the owning device clears it or the programme acknowledges it.
- BASIC is best for polling and setup. Keep real interrupt handlers in machine code.
- VBlank is a frame event. It is not a calibrated timer.
- WAIT\_VBLANK is tied to presentation. If presentation is unavailable, the wait path has a safety timeout so a programme can continue.

Chapter 41 uses these timing rules when it lays out larger programmes and their buffers.